SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

Duration : 45 Minutes

QUESTIONS: 2 **Total Marks:20**

Annexure A

INTEGRATED EXPERIMENTS

Code of Conduct:

I __ATHIN SIVA.S_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

XPERIMENT 1: DECISION TREE CLASSIFICATION

Objective

To implement and evaluate a Decision Tree classifier on a given dataset such as the Iris or Play-Tennis dataset using Python and an appropriate machine-learning library. The experiment demonstrates data preprocessing, train-test splitting, Decision Tree construction using Gini Index or Information Gain, prediction and evaluation using standard classification metrics.

1. Aim

To build a Decision Tree classification model, visualize the first five levels of the tree, evaluate its performance using accuracy, precision, recall and F1-score, and identify misclassified instances.

2. Software / Requirements

Requirement	Details
Programming Language	Python 3.x
Libraries	Pandas, NumPy, Scikit-learn, Matplotlib
Dataset	Iris dataset (built into scikit-learn)
Classifier	DecisionTreeClassifier
Splitting Criterion	Gini Index or Entropy / Information Gain
Environment	Python IDLE, Jupyter Notebook, Google Colab or VS Code

3. Theory

A Decision Tree is a supervised learning algorithm used for classification and regression. For classification, the algorithm recursively divides the dataset into smaller groups using feature-based conditions. Each internal node represents a decision, each branch represents an outcome of that decision and each leaf represents a predicted class.

The root node is selected using a splitting criterion. Two commonly used criteria are Gini Index and Entropy.

Gini Index

$Gini = 1 - \Sigma(p_i^2)$

For a node containing two or more classes, pi is the proportion of samples belonging to class i. A Gini value of 0 indicates a completely pure node.

Entropy

$Entropy(S) = -\Sigma\ p_i \log_2(p_i)$

Entropy measures uncertainty or impurity. Lower entropy indicates a purer node.

Information Gain

Information Gain = Entropy(parent) − Weighted Entropy(children)

The feature producing the largest reduction in impurity is preferred for splitting.

4. Dataset – Iris

The Iris dataset contains 150 samples, four numerical features and three classes. Each class contains 50 samples.

Feature	Description
Sepal Length	Length of the sepal in centimetres
Sepal Width	Width of the sepal in centimetres
Petal Length	Length of the petal in centimetres
Petal Width	Width of the petal in centimetres
Target	Setosa, Versicolor or Virginica

5. Step-by-Step Procedure

Import the required Python libraries.

Load the Iris dataset using scikit-learn.

Create a Pandas DataFrame for easier inspection.

Check the dataset dimensions and identify missing values.

Separate the input features X from the target y.

Split the dataset into training and testing sets using a stratified 70:30 split.

Create a DecisionTreeClassifier using criterion='gini' or criterion='entropy'.

Train the classifier using the training data.

Visualize the first five levels of the tree.

Predict the classes of the test samples.

Calculate accuracy, precision, recall and F1-score.

Generate a confusion matrix and classification report.

Identify at least two misclassified test instances, if present.

State the final result and conclusion.

6. Python Program

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, precision_score, recall_score
from sklearn.metrics import f1_score, classification_report, confusion_matrix

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Create DataFrame
df = pd.DataFrame(X, columns=iris.feature_names)
df["target"] = y

print("Dataset Shape:", df.shape)
print("\nMissing Values:")
print(df.isnull().sum())

# Train-test split: 70:30
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.30, random_state=42, stratify=y
)

# Create Decision Tree
model = DecisionTreeClassifier(
criterion="gini",
max_depth=5,
random_state=42
)

# Train
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Evaluation
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average="weighted")
recall = recall_score(y_test, y_pred, average="weighted")
f1 = f1_score(y_test, y_pred, average="weighted")

print("\nAccuracy:", round(accuracy, 4))
print("Precision:", round(precision, 4))
print("Recall:", round(recall, 4))
```

```
print("F1-Score:", round(f1, 4))

print("\nClassification Report:")
print(classification_report(y_test, y_pred,
target_names=iris.target_names))

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Plot first five levels
plt.figure(figsize=(16, 9))
plot_tree(
model,
feature_names=iris.feature_names,
class_names=iris.target_names,
filled=True,
max_depth=4
)
plt.title("Decision Tree - First Five Levels")
plt.show()

# Find misclassified instances
misclassified = np.where(y_test != y_pred)[0]

print("\nMisclassified test instances:")
for i in misclassified:
print("Test index:", i,
"Actual:", iris.target_names[y_test[i]],
"Predicted:", iris.target_names[y_pred[i]])
```

7. Explanation of the Program

Step	Explanation
Import libraries	Loads tools for data handling, visualization, model building and evaluation.
Load Iris	Loads the standard Iris classification dataset.
Check missing values	Confirms whether preprocessing is required for missing data.
Train-test split	Uses 70% data for training and 30% for testing.
Create model	Creates a Decision Tree with Gini Index and maximum depth 5.
Fit model	Learns decision rules from training samples.
Predict	Classifies unseen test samples.
Evaluate	Computes accuracy, precision, recall and F1-score.
Visualize	Displays the first five levels of the Decision Tree.
Misclassification	Compares actual and predicted labels to find errors.

8. Sample Output

A typical run on the Iris dataset gives very high classification performance. Exact values can vary if the split, criterion or tree parameters are changed.

Dataset Shape: (150, 5)

Missing Values: 0 for every feature

Example evaluation:

Metric	Example Result
Accuracy	0.9333
Weighted Precision	0.9340 (approximately)
Weighted Recall	0.9333
Weighted F1-Score	0.9330 (approximately)

The exact classification report and confusion matrix should be copied from the user's execution environment because model settings and random seeds affect the final test predictions.

9. Misclassified Instances

The program automatically identifies misclassified test instances. For example, an instance may belong to the actual class Versicolor but be predicted as Virginica because the two classes have overlapping feature values. At least two misclassified instances should be listed when the executed model produces two or more errors.

Format for reporting:

1. Test index: _____ | Actual class: _____ | Predicted class: _____

2. Test index: _____ | Actual class: _____ | Predicted class: _____

10. Result and Discussion

The Decision Tree classifier was successfully trained on the Iris dataset and evaluated on unseen test samples. The model achieved high classification performance because the Iris classes, particularly Setosa, are relatively well separated using petal measurements. The tree is also interpretable because each path from the root to a leaf represents a sequence of decision rules.

The first levels of the tree generally emphasize petal-related measurements because these features provide strong separation among Iris species. Pruning or limiting the tree depth helps prevent unnecessary complexity and reduces the risk of overfitting.

11. Conclusion

The experiment demonstrates the complete workflow of Decision Tree classification: dataset loading, preprocessing, train-test splitting, model construction, tree visualization, prediction and evaluation. Gini Index or Information Gain can be used to select effective splits. The Decision Tree provides both good predictive performance and an interpretable structure, making it a useful introductory model for classification.

EXPERIMENT 2: REINFORCEMENT LEARNING – Q-LEARNING

Objective

To implement Q-Learning for a simple 4×4 Grid World environment, observe the agent's learning process, record Q-values at selected episodes, extract the final learned policy and plot cumulative reward per episode to study convergence.

1. Aim

To train a Q-Learning agent in a 4×4 Grid World using states, actions and rewards, and to demonstrate how repeated interaction with the environment allows the agent to learn an optimal or near-optimal path to the goal.

2. Grid World Environment

The environment contains 16 states arranged in a 4×4 grid. The agent starts from a fixed starting state and attempts to reach a goal state. Four actions are allowed: Up, Down, Left and Right.

The state numbering used in this experiment is:

0 1 2 3
4 5 6 7
8 9 10 11
12 13 14 15

Example: state 0 is the start and state 15 is the goal.

3. States and Actions

Component	Definition
States	0 to 15, representing the 16 grid cells
Actions	0 = Up, 1 = Down, 2 = Left, 3 = Right
Start	State 0
Goal	State 15
Normal movement reward	−1

Goal reward	+10
Invalid/out-of-grid move	−1 and agent remains in current state
Terminal state	State 15

4. Q-Learning Theory

Q-Learning is a model-free Reinforcement Learning algorithm. The agent learns a Q-value for every state-action pair. The Q-value estimates the expected long-term reward obtained by taking an action in a particular state and then following an optimal policy.

The Q-Learning update rule is:

$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max Q(s',a') - Q(s,a)]$

where:

s = current state

a = current action

r = immediate reward

s′ = next state

α = learning rate

γ = discount factor

max Q(s′,a′) = best estimated future value

An epsilon-greedy strategy is used to balance exploration and exploitation. With probability ε, the agent explores a random action; otherwise it chooses the action with the highest Q-value.

5. Suggested Parameters

Parameter	**Value**
Grid size	4 × 4
Number of states	16
Number of actions	4
Learning rate α	0.8
Discount factor γ	0.95
Initial epsilon	1.0
Minimum epsilon	0.05
Epsilon decay	0.995
Episodes	100
Maximum steps per episode	100
Goal reward	+10
Movement reward	−1

6. Step-by-Step Procedure

Create a 4×4 Grid World with 16 states.

Define the start state and goal state.

Define Up, Down, Left and Right actions.

Assign a reward of +10 for reaching the goal and −1 for ordinary or invalid moves.

Initialize a Q-table with zeros. Its size is 16 × 4.

Set learning rate, discount factor and epsilon values.

For each episode, reset the agent to the start state.

Choose an action using epsilon-greedy selection.

Move to the next state and receive a reward.

Update the Q-table using the Q-Learning equation.

Continue until the goal is reached or the maximum step count is exceeded.

Record cumulative reward for each episode.

Print Q-values at episodes 1, 50 and 100.

Extract the final policy using the action with maximum Q-value at each state.

Plot cumulative reward versus episode number and explain convergence.

7. Python Program

```
import numpy as np
import matplotlib.pyplot as plt

# 4x4 Grid World
GRID_SIZE = 4
N_STATES = 16
N_ACTIONS = 4

START = 0
GOAL = 15

# Actions: 0=Up, 1=Down, 2=Left, 3=Right
ACTIONS = {
0: (-1, 0),
1: (1, 0),
2: (0, -1),
3: (0, 1)
}

alpha = 0.8
gamma = 0.95
epsilon = 1.0
epsilon_min = 0.05
epsilon_decay = 0.995

episodes = 100
max_steps = 100

Q = np.zeros((N_STATES, N_ACTIONS))
episode_rewards = []

def step(state, action):
row, col = divmod(state, GRID_SIZE)
dr, dc = ACTIONS[action]

new_row = row + dr
new_col = col + dc

# Invalid move
if new_row < 0 or new_row >= GRID_SIZE or new_col < 0 or new_col >= GRID_SIZE:
return state, -1, False

next_state = new_row * GRID_SIZE + new_col

# Goal
if next_state == GOAL:
return next_state, 10, True

return next_state, -1, False

for episode in range(1, episodes + 1):
state = START
total_reward = 0

for step_no in range(max_steps):
# Epsilon-greedy action
if np.random.rand() < epsilon:
action = np.random.randint(N_ACTIONS)
```

```
else:
action = np.argmax(Q[state])

next_state, reward, done = step(state, action)

# Q-Learning update
best_next = np.max(Q[next_state])
Q[state, action] = Q[state, action] + alpha * (
reward + gamma * best_next - Q[state, action]
)

state = next_state
total_reward += reward

if done:
break

episode_rewards.append(total_reward)

epsilon = max(epsilon_min, epsilon * epsilon_decay)

if episode in [1, 50, 100]:
print("\nEpisode:", episode)
print(np.round(Q, 2))

# Final policy
action_symbols = {0: "↑", 1: "↓", 2: "←", 3: "→"}

print("\nFinal Learned Policy:")
for state in range(N_STATES):
if state == GOAL:
symbol = "G"
else:
symbol = action_symbols[np.argmax(Q[state])]
print(symbol, end=" ")
if (state + 1) % GRID_SIZE == 0:
print()

# Plot cumulative reward
plt.figure(figsize=(10, 5))
plt.plot(episode_rewards)
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Q-Learning Convergence")
plt.grid(True)
plt.show()
```

8. Explanation of the Q-Learning Code

Code Part	Purpose
Q-table	Stores four Q-values for each of the 16 states.
step()	Calculates the next state and reward.
epsilon-greedy	Balances exploration and exploitation.
Q update	Updates the value of the current state-action pair.
Episode loop	Repeats learning over 100 episodes.
Reward list	Stores cumulative reward for convergence analysis.
Final policy	Chooses the action with the largest Q-value at each state.

Reward plot	Shows whether learning improves over episodes.

9. Q-Table Structure

The Q-table contains 16 rows and 4 columns.

State	Up	Down	Left	Right
0	Q(0,U)	Q(0,D)	Q(0,L)	Q(0,R)
1	Q(1,U)	Q(1,D)	Q(1,L)	Q(1,R)
...	...	...	...	...
14	Q(14,U)	Q(14,D)	Q(14,L)	Q(14,R)
15	Terminal	Terminal	Terminal	Terminal

10. Recording Q-Values at Episodes 1, 50 and 100

The program prints the complete Q-table at Episodes 1, 50 and 100. Because Q-Learning contains random exploration, exact numerical values can differ from one execution to another. Therefore, the values printed by the student's execution should be used in the final laboratory record.

Episode	Expected Learning Stage	What to Observe
1	Initial learning	Most Q-values are close to zero because the agent has little experience.
50	Intermediate learning	States closer to the goal generally begin to develop larger positive values and the path becomes more consistent.
100	Final learning	A clear path toward the goal should emerge and Q-values along useful actions should be higher.

11. Extracting the Final Learned Policy

The final policy is obtained using:

$\pi(s) = \arg\max_a Q(s,a)$

This means the agent chooses the action with the largest Q-value for each state.

For a 4×4 grid where the shortest path is unobstructed and the goal is state 15, an ideal learned policy may resemble:

→ → → ↓
↓ ↓ ↓ ↓
→ → → ↓
→ → → G

This is an example of an optimal shortest-path policy. The actual policy produced by the program should be reported after execution because exploration and parameter settings can affect the learned Q-table.

12. Cumulative Reward and Convergence

Cumulative reward is the sum of all rewards received during an episode. In this environment, every ordinary move has a reward of −1 and reaching the goal provides +10. Therefore, an agent that reaches the goal using fewer steps receives a better total reward.

At the beginning, the agent explores many actions and may hit boundaries or take long paths, producing low or negative rewards. As learning progresses, the Q-values improve and the agent increasingly chooses actions leading toward the goal. Consequently, cumulative reward should generally improve and become more stable.

A convergence trend does not necessarily mean that the Q-table has reached the mathematically exact optimum. It means that the learned behavior becomes relatively stable under the selected parameters.

13. Result

The Q-Learning agent was successfully implemented for a 4×4 Grid World. The agent initially explored the environment randomly and gradually learned useful state-action values through repeated interaction. By the later

episodes, the policy should favor movements that reduce the distance to the goal. The cumulative reward plot provides visual evidence of learning and convergence.

14. Conclusion

This experiment demonstrates the fundamental Reinforcement Learning cycle: observe the current state, select an action, receive a reward, move to a new state and update the Q-value. Unlike supervised learning, the agent is not given the correct action for every state. Instead, it learns through trial and error. The final Q-table represents the agent's learned knowledge, while the extracted policy shows the preferred action at each state.

15. Viva / Important Questions

Question	Answer
What is a Decision Tree?	A supervised learning model that makes predictions through a sequence of feature-based decisions.
What is Gini Index?	A measure of node impurity used to choose Decision Tree splits.
What is Information Gain?	The reduction in entropy obtained after splitting a dataset.
What is Q-Learning?	A model-free RL algorithm that learns state-action values from rewards.
What is a Q-table?	A table storing the estimated value of every state-action pair.
What is epsilon-greedy?	A strategy that balances random exploration with selecting the best-known action.
What is alpha?	The Q-Learning learning rate.
What is gamma?	The discount factor controlling the importance of future rewards.
Why does reward improve?	The agent learns Q-values that favor actions leading to the goal with fewer penalties.
Why can exact Q-values differ?	Random exploration and stochastic execution can produce different learning paths.

OVERALL LEARNING OUTCOME

The two integrated experiments demonstrate two important AI paradigms. Decision Tree Classification is a supervised learning approach in which the model learns from labeled examples and constructs interpretable decision rules. Q-Learning is a Reinforcement Learning approach in which an agent learns from interaction, rewards and repeated exploration. Together, the experiments provide practical understanding of classification, impurity-based splitting, model evaluation, state-action representation, reward design, Q-value updates, policy extraction and convergence analysis.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation